

ClusterRole in Kubernetes

Since you're learning **KCNA as a beginner**, let's understand **ClusterRole** very simply.

1. What is a ClusterRole?

A ClusterRole is a Kubernetes RBAC object that defines a set of permissions at the cluster level.

In simple words:

ClusterRole = A collection of permissions that can be used across the Kubernetes cluster.

 **Easy memory:**

ClusterRole = Permissions for the cluster

2. Why Do We Need ClusterRole?

Suppose you have an administrator who needs to:

View Nodes

View Pods

View Services

View Deployments

Instead of creating separate permissions one by one, you can put them into a **ClusterRole**.

ClusterRole

|

├— View Nodes

├— View Pods

├— View Services

└— View Deployments

Then you can give that ClusterRole to a user using a **ClusterRoleBinding**.

3. ClusterRole vs Role

This is very important.

Role

A **Role** is normally used for permissions within **one namespace**.

Namespace A

|

└─ Role

└─ Can view Pods

ClusterRole

A **ClusterRole** can define permissions that apply at the **cluster level**, and it can also be used to grant access to namespaced resources across multiple namespaces when combined with the appropriate binding.

Kubernetes Cluster

|

├─ Namespace A

├─ Namespace B

└─ Namespace C

↑

ClusterRole

Easy memory:

Role → Namespace

ClusterRole → Cluster

4. Real-Life Example 🏡

Imagine a school.

Role

A teacher has permission to manage students in **Class 5 only**.

Class 5



Teacher Role

ClusterRole

The principal can view information from **the whole school**.

Whole School



ClusterRole

So:

Role = limited area

ClusterRole = broader/cluster-level permissions

5. ClusterRole Does NOT Give Permission Automatically

This is an important point.

Creating:

ClusterRole

doesn't automatically give permissions to a user.

You need a **binding**.

User



ClusterRoleBinding



ClusterRole

|



Permissions

6. ClusterRoleBinding

What is it?

ClusterRoleBinding connects a ClusterRole to a user, group, or service account.

Example:

Abeera

|



ClusterRoleBinding

|



ClusterRole

|



Permissions

So remember:

ClusterRole

 **Defines WHAT permissions exist.**

ClusterRoleBinding

 **Connects WHO gets those permissions.**

7. Example

Suppose we create a ClusterRole called:

pod-reader

It allows:

get Pods

list Pods

watch Pods

Conceptually:

ClusterRole: pod-reader

Permissions:

✓ get pods

✓ list pods

✓ watch pods

Then:

User: developer

|



ClusterRoleBinding

|



pod-reader

Now the developer receives those permissions.

8. ClusterRole YAML Example

apiVersion: rbac.authorization.k8s.io/v1

kind: ClusterRole

metadata:

name: pod-reader

rules:

- apiGroups: [""]

resources: ["pods"]

verbs: ["get", "list", "watch"]

The important part is:

resources:

- pods

and:

verbs:

- get

- list

- watch

Meaning:

This ClusterRole allows reading Pods.

9. ClusterRoleBinding Example

apiVersion: rbac.authorization.k8s.io/v1

kind: ClusterRoleBinding

metadata:

name: pod-reader-binding

subjects:

- kind: User

name: abeera

roleRef:

kind: ClusterRole

name: pod-reader

apiGroup: rbac.authorization.k8s.io

This connects:

User: abeera



ClusterRoleBinding



ClusterRole: pod-reader

10. Important: ClusterRole Can Be Used with Namespaced Resources

This is a slightly advanced but **important KCNA concept**.

A ClusterRole can define permissions for resources such as:

Pods

Services

Deployments

When a **ClusterRoleBinding** is used, those permissions can apply across the cluster.

For example:

ClusterRole



Pod read permissions



ClusterRoleBinding



User



Can read Pods across namespaces

11. ClusterRole Can Also Be Used with RoleBinding

A ClusterRole can also be granted **within a specific namespace** using a RoleBinding.

Example:

ClusterRole



RoleBinding



Namespace A

This means the ClusterRole's permissions are granted only within that namespace through that RoleBinding.

This is a useful distinction:

ClusterRole + ClusterRoleBinding

→ Cluster-wide access

ClusterRole + RoleBinding

→ Access limited to the RoleBinding's namespace

12. Useful Commands ★

See ClusterRoles

`kubectl get clusterroles`

Short form:

`kubectl get clusterrole`

Describe a ClusterRole

`kubectl describe clusterrole pod-reader`

See ClusterRoleBindings

`kubectl get clusterrolebindings`

Describe a ClusterRoleBinding

`kubectl describe clusterrolebinding pod-reader-binding`

Create a ClusterRole

For example:

`kubectl create clusterrole pod-reader \`

`--verb=get,list,watch \`

`--resource=pods`

Create a ClusterRoleBinding

`kubectl create clusterrolebinding pod-reader-binding \`

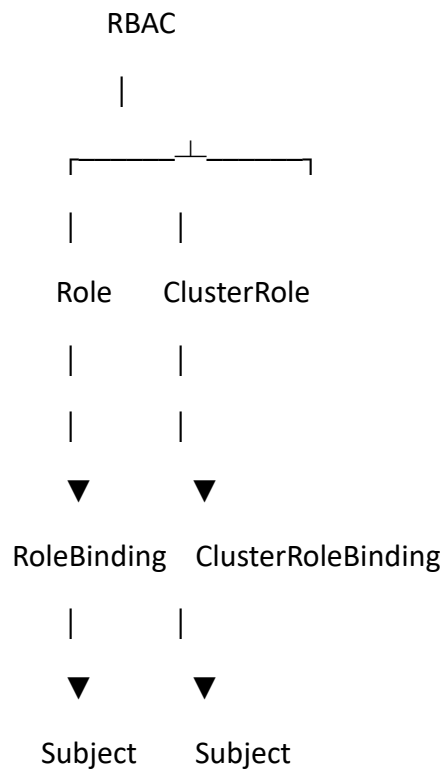
`--clusterrole=pod-reader \`

`--user=abeera`

13. Role vs ClusterRole vs Bindings ★★ ★

Object	Purpose
Role	Defines permissions within a namespace
ClusterRole	Defines cluster-level permissions / reusable permission sets
RoleBinding	Gives a Role or ClusterRole to a subject within a namespace
ClusterRoleBinding	Gives a ClusterRole to a subject across the cluster

Easy diagram:



KCNA Revision Notes

ClusterRole

- Part of **RBAC**.
- Defines a set of permissions.
- Has **cluster-level scope**.
- Can define permissions for cluster-scoped resources.

- Can also be used for namespaced resources.
- Does **not** grant permissions by itself.
- Usually combined with a **binding**.

ClusterRoleBinding

- Connects a **ClusterRole** to a user, group, or service account.
- Gives the permissions defined by the ClusterRole.
- ClusterRoleBinding grants those permissions **cluster-wide**.

★ Most Important Memory Trick

ClusterRole



WHAT can you do?



Permissions 

ClusterRoleBinding



WHO gets those permissions?



User / Group / ServiceAccount 

One-line definition:

ClusterRole defines a set of RBAC permissions that can be used at the cluster level, while ClusterRoleBinding grants those permissions to a subject across the cluster.